

Sprawozdanie z pracowni specjalistycznej

Systemy wbudowane

Tytuł: *Zatrzaski, przerzutniki, rejestry*

Wykonujący: *Monika Słowikowska, Filip Rutkowski*

Numer grupy PS: 9

Numer zespołu: 6

Numer pracy laboratoryjnej: 5

Studia dzienne

Kierunek: Informatyka

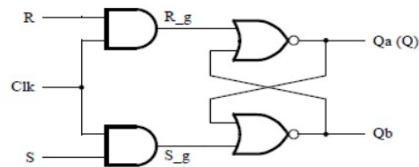
Semestr: VI

Prowadzący ćwiczenie: prof. dr hab. inż. Salauyou Valery

1. Zadanie 1

Utwórz dwa projekty zatrzasków RS z rys. 1.

Utwórz dwa projekty zatrzasków RS z rys. 1.



Rys. 1. Układ zatrzasku RS utworzony z bramek logicznych

latch RS_gates (rys. 2) używając prymitywów bramek języka Verilog, a drugi latch rys. 3) za pomocą wyrażeń logicznych.

Jeden projekt latch RS_gates (rys. 2) używając prymitywów bramek języka Verilog, a drugi latch RS_boolean (rys. 3) za pomocą wyrażeń logicznych.

```
// bramkowy zatrzask RS
module RS_gates (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb    /* synthesis keep */;
    and (R_g, R, Clk);
    and (S_g, S, Clk);
    nor (Qa, R_g, Qb);
    nor (Qb, S_g, Qa);
    assign Q = Qa;
endmodule
```

Rys. 2. Opis zatrzasku RS, poprzez wykorzystanie instancji elementów logicznych

```
// bramkowy zatrzask RS
module RS_boolean (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb /* synthesis keep */;
    assign R_g = R & Clk;
    assign S_g = S & Clk;

    assign Qa = ~(R_g | Qb);
    assign Qb = ~(S_g | Qa);
    assign Q = Qa;
endmodule
```

Rys. 3. Definicja zatrzasku RS przy użyciu wyrażeń logicznych

Wykorzystaj dyrektywę keep Aby monitorować wewnętrzne węzły projektu w symulacji. Porównaj wyniki syntezy (RTL Viewer, Technology Map Viewer) bez używania i z użyciem dyrektywy keep.

Wskazówka: Symulator nie daje komunikatu o błędzie z początkowymi wartościami CLK = 1 i R = 1. Sprawdź, co podaje symulator dla R = 1 i S = 1. Zaimplementuj projekt na płycie z następującymi pinami:

Wyprowadzenia	Piny
clk	SW[2]
R	SW[1]
S	SW[0]
Q	LEDR[0]

Kod w języku Verilog:

```
module latch_RS (  
    input [2:0] SW,  
    output [0:0] LEDR  
);  
    RS_boolean ext(SW[2], SW[1], SW[0], LEDR[0]);  
endmodule
```

Główny moduł, w którym wywoływane były pozostałe moduły (np. RS_boolean, jak na zaprezentowanym wycinku kodu).

```

module RS_boolean (Clk, R, S, Q);
input Clk, R, S;
output Q;

assign R_g = R & Clk;
assign S_g = S & Clk;
assign Qa = ~(R_g | Qb);
assign Qb = ~(S_g | Qa);
assign Q = Qa;

endmodule

```

Moduł RS_boolean, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych i przypisania ciągłego (assign) bez dyrektywy keep

```

module RS_boolean_keep (Clk, R, S, Q);
input Clk, R, S;
output Q;

wire R_g, S_g, Qa, Qb /* synthesis keep */ ;
assign R_g = R & Clk;
assign S_g = S & Clk;
assign Qa = ~(R_g | Qb);
assign Qb = ~(S_g | Qa);
assign Q = Qa;

endmodule

```

Moduł RS_boolean_keep, czyli przerzutnik RS zbudowany przy użyciu użyciu wyrażeń logicznych, przypisania ciągłego (assign) oraz dyrektywy keep.

```

module RS_gates (Clk, R, S, Q);
input Clk, R, S;
output Q;

and (R_g, R, Clk);
and (S_g, S, Clk);
nor (Qa, R_g, Qb);
nor (Qb, S_g, Qa);

assign Q = Qa;

endmodule

```

Moduł RS_gates, czyli przerzutnik RS zbudowany na bramkach bez dyrektywy keep.

```

module RS_gates_keep (Clk, R, S, Q);
input Clk, R, S;
output Q;

wire R_g, S_g, Qa, Qb /* synthesis keep */ ;
and (R_g, R, Clk);
and (S_g, S, Clk);
nor (Qa, R_g, Qb);
nor (Qb, S_g, Qa);

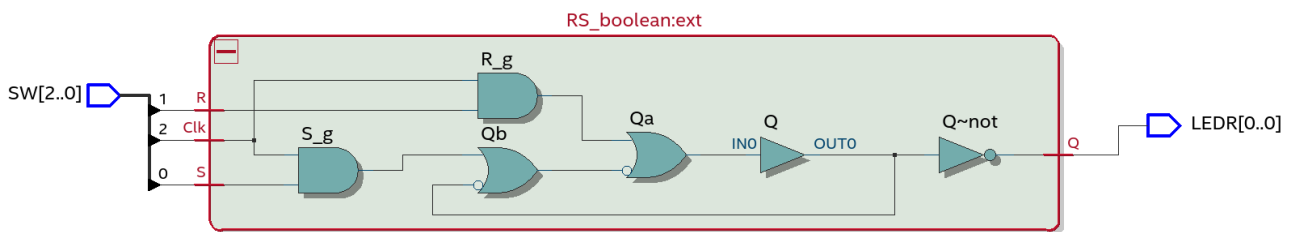
assign Q = Qa;

endmodule

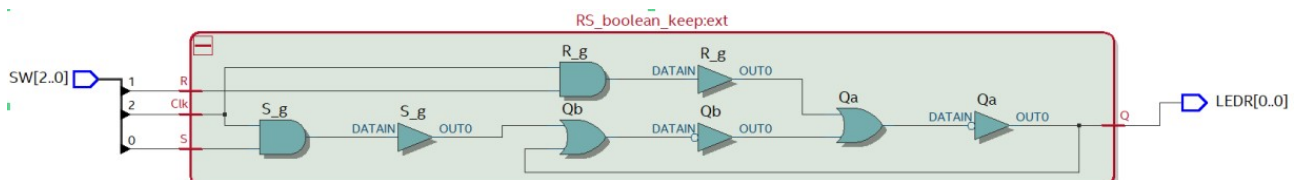
```

Moduł RS_gates_keep, czyli przerzutnik RS zbudowany na bramkach przy zastosowaniu dyrektywy keep.

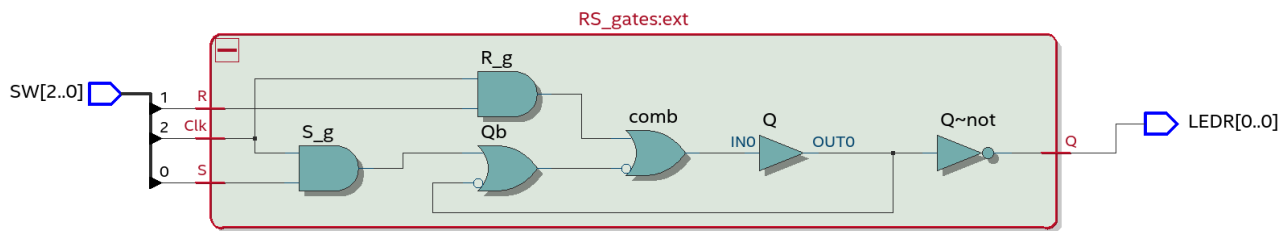
Wyniki RTL Viewer:



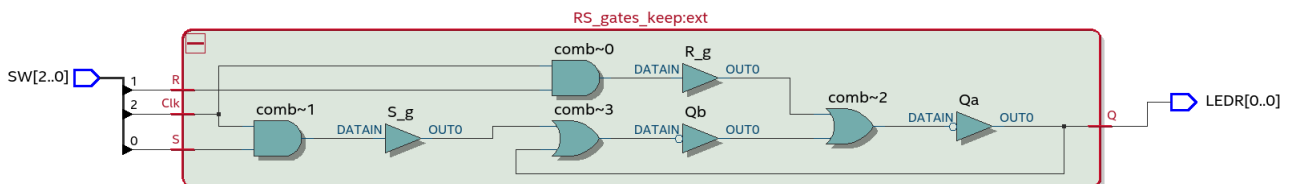
Moduł RS_boolean, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych i przypisania ciągłego (assign) bez dyrektywy keep.



Moduł RS_boolean_keep, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych, przypisania ciągłego (assign) oraz dyrektywy keep.

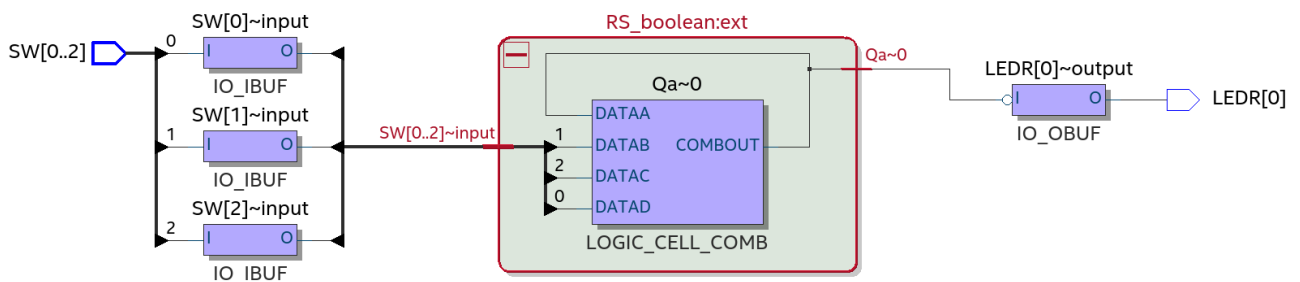


Moduł RS_gates, czyli przerzutnik RS zbudowany na bramkach bez dyrektywy keep.

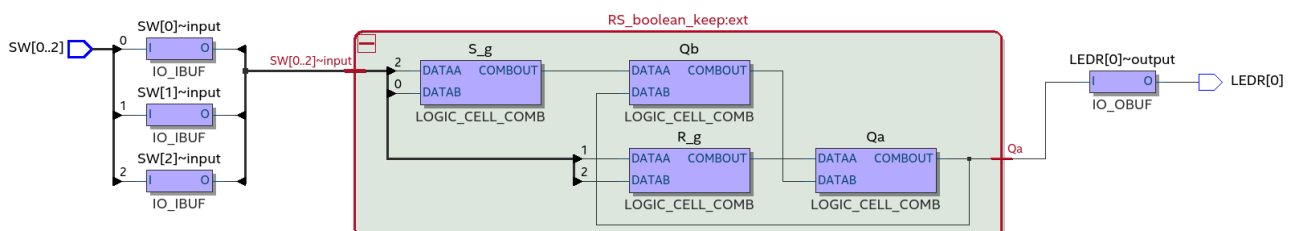


Moduł RS_gates_keep, czyli przerzutnik RS zbudowany na bramkach przy zastosowaniu dyrektywy keep.

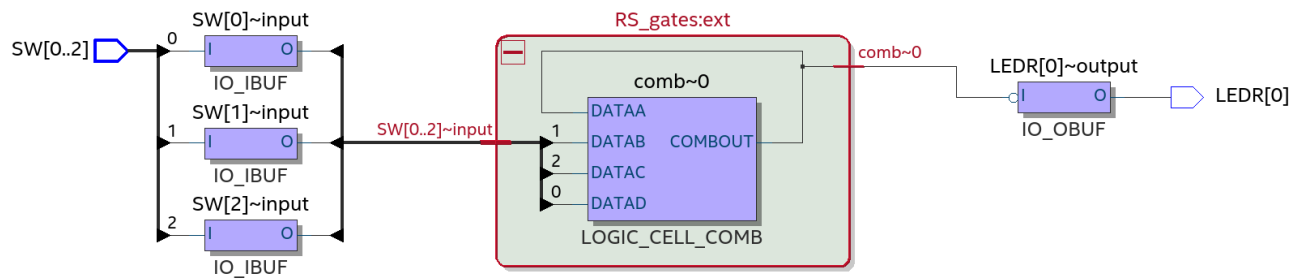
Wyniki Technology Map Viewer:



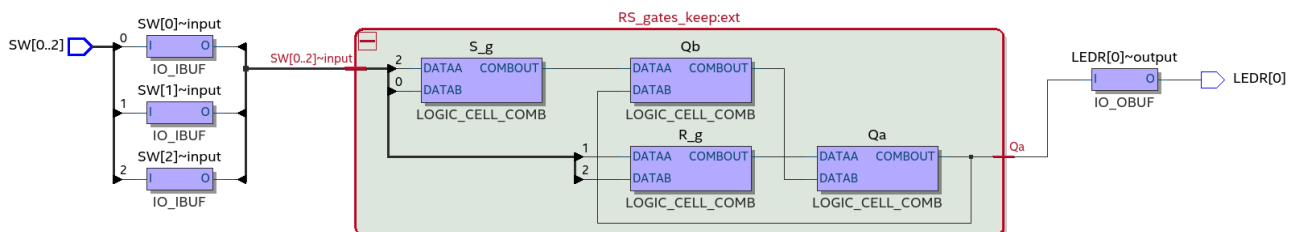
Moduł RS_boolean, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych i przypisania ciągłego (assign) bez dyrektywy keep.



Moduł RS_boolean_keep, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych, przypisania ciągłego (assign) oraz dyrektywy keep.

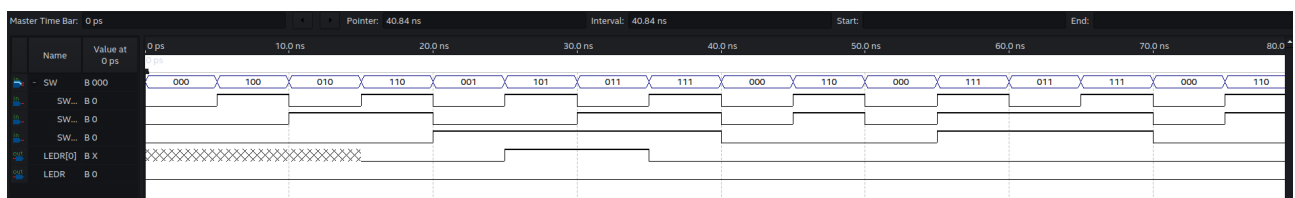


Moduł RS_gates, czyli przerzutnik RS zbudowany na bramkach bez dyrektywy keep.

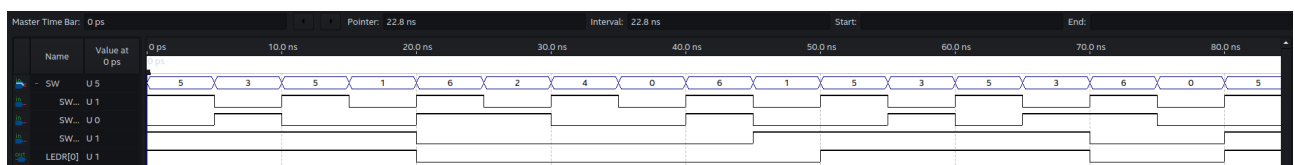


Moduł RS_gates_keep, czyli przerzutnik RS zbudowany na bramkach przy zastosowaniu dyrektywy keep.

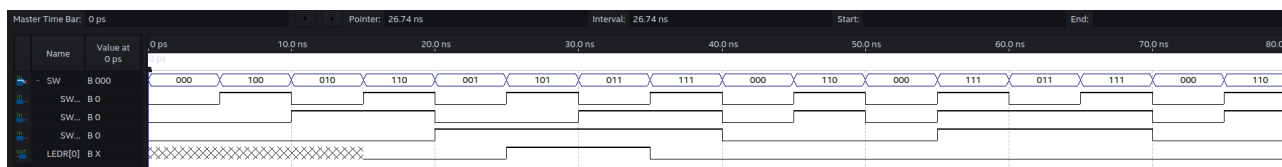
Wyniki symulacji:



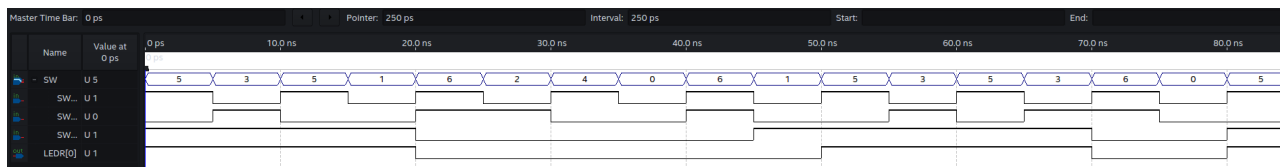
Symulacja dla modułu RS_boolean.



Symulacja dla modułu RS_boolean_keep.



Symulacja dla modułu RS_gates.



Symulacja dla modułu RS_gates_keep.

Wnioski:

Zgodnie z poleceniem przetestowaliśmy cztery różne moduły opisujące ten sam układ: przerzutnik typu RS. Były to kolejno:

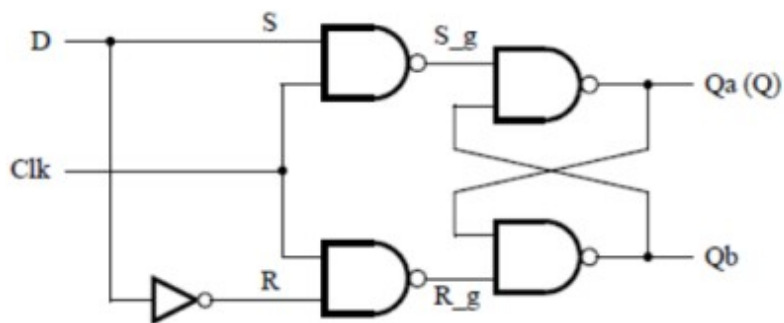
- moduł **RS_gates**, czyli przerzutnik RS zbudowany na bramkach bez dyrektywy keep,
- moduł **RS_boolean**, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych, i przypisania ciągłego (assign) bez dyrektywy keep,
- moduł **RS_gates_keep**, czyli przerzutnik RS zbudowany na bramkach przy zastosowaniu dyrektywy keep,
- moduł **RS_boolean_keep**, czyli przerzutnik RS zbudowany przy użyciu wyrażeń logicznych, przypisania ciągłego (assign) oraz dyrektywy keep.

Na podstawie wyżej zamieszczonych schematów jesteśmy w stanie stwierdzić, że dodanie dyrektywy keep wymusza wygenerowanie dodatkowych kilku bramek odpowiadających za monitorowanie węzłów.

2. Zadanie 2

Utwórz projekt latch_D (rys. 4), używając dowolnego ze stylów opisu (używając prymitywów lub wyrażeń logicznych). Wykonaj wszystkie kroki z poprzedniego zadania dla tego projektu. Przydzielenie pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Q	LEDR[0]



Rys. 4. Schemat zatrzasku D

Kod w języku Verilog:

```
module latch_D (
    input [1:0] SW,
    output [0:0] LEDR);

    latch_D_boolean ex(SW[1], SW[0], LEDR[0]);

endmodule
```

Główny moduł.

```
module latch_D_boolean(Clk, D, Q);
    input Clk, D;
    output Q;

    wire D_g1, D_g2, Qa, Qb ;
    assign D_g1 = ~(D & Clk);
    assign D_g2 = ~(~D & Clk);
    assign Qa = ~(D_g1 & Qb);
    assign Qb = ~(D_g2 & Qa);
    assign Q = Qa;
endmodule
```

Przerzutnik typu D bez dyrektywy keep.

```

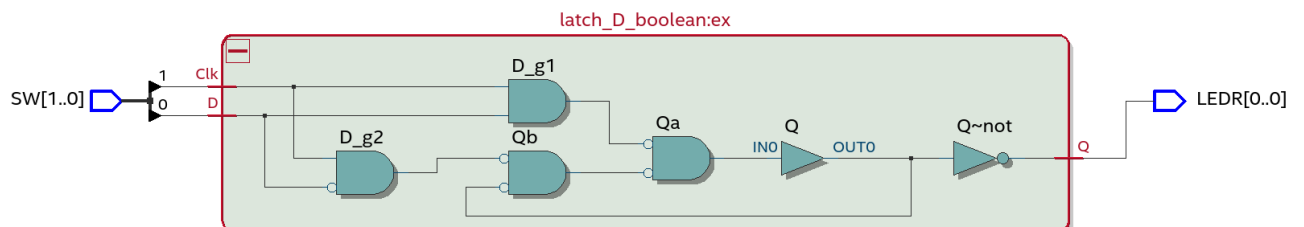
module latch_D_boolean_keep(Clk, D, Q);
    input Clk, D;
    output Q;

    wire D_g1, D_g2, Qa, Qb /* synthesis keep */ ;
    assign D_g1 = ~(D & Clk);
    assign D_g2 = ~(~D & Clk);
    assign Qa = ~(D_g1 & Qb);
    assign Qb = ~(D_g2 & Qa);
    assign Q = Qa;
endmodule

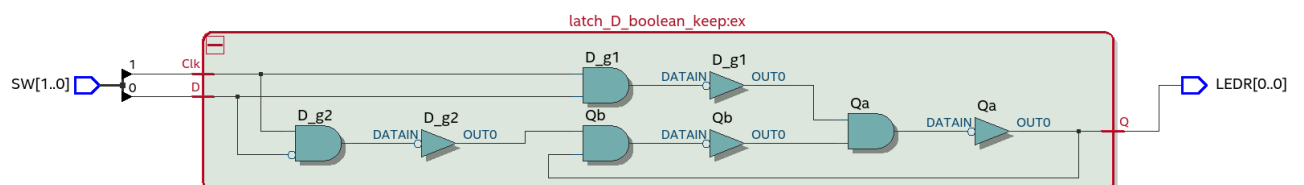
```

Przerzutnik typu D z dyrektywą keep.

Wyniki RTL Viewer:

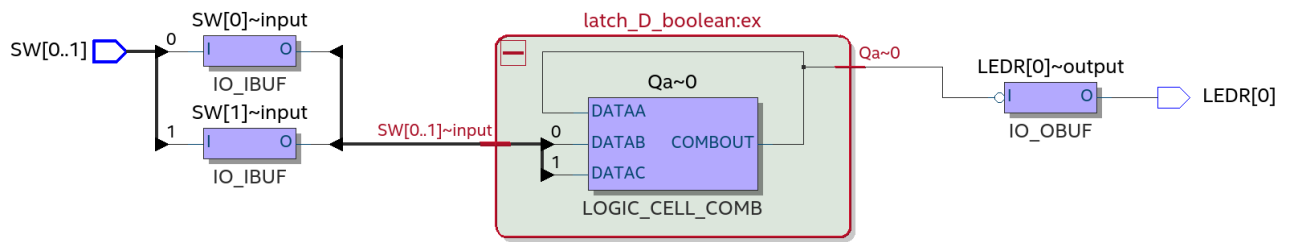


Schemat dla modułu bez dyrektywy keep.

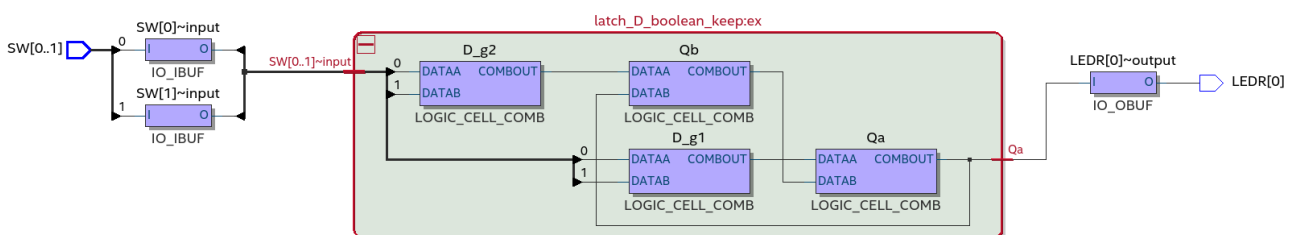


Schemat dla modułu z dyrektywą keep.

Wyniki Technology Map Viewer:

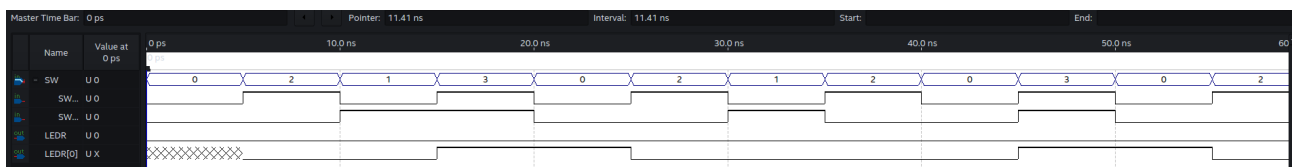


Schemat dla modułu bez dyrektywy keep.

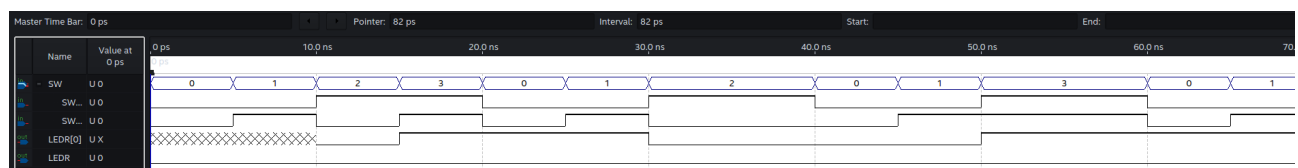


Schemat dla modułu z dyrektywą keep.

Wyniki symulacji:



Symulacja działania (bez dyrektywy keep).



Symulacja działania (z dyrektywą keep).

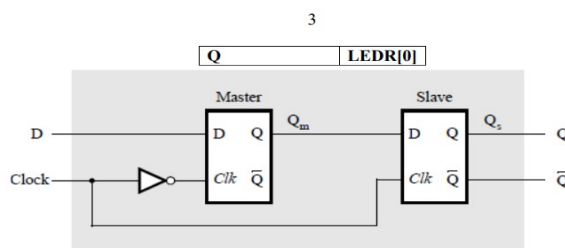
Wnioski:

Utworzyliśmy projekt D_latch przy użyciu wyrażeń logicznych w module D_latch_boolean opisującym przerzutnik typu D. Wygenerowaliśmy widoki RTL, Map Viewer oraz symulacje odpowiednio dla wariantu z dyrektywą keep oraz bez niej. Na wygenerowanych schematach widać, że moduł z dyrektywą keep ma dodatkowe bramki do monitorowania węzłów.

3. Zadanie 3

Utwórz projekt master_slave_D (rysunek 5) przerzutnika D z dwóch zatrzasków. Użyj dyrektywy keep do monitorowania wewnętrznych węzłów. Porównaj wyniki syntezy (RTL Viewer, Technology Map Viewer) bez użycia i korzystania z dyrektywy keep. Uruchom symulację. Przy jakich wartościach początkowych symulator wyświetla komunikat o błędzie? Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Q	LEDR[0]



Rys. 5. Schemat przerzutnika D master-slave



Kod w języku Verilog:

```

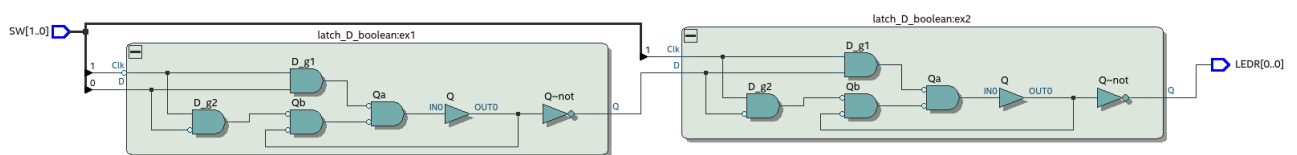
]module master_slave_D (
    input[1:0] SW,
    output[0:0] LEDR);

    wire Qm;

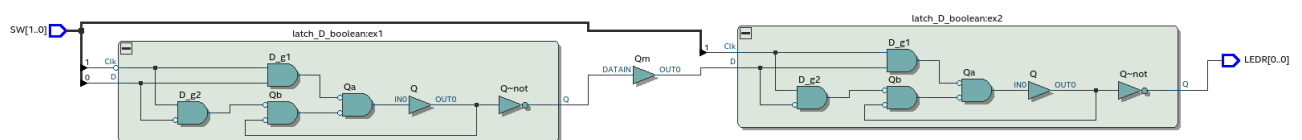
    latch_D_boolean ex1(~SW[1], SW[0], Qm);
    latch_D_boolean ex2(SW[1], Qm, LEDR[0]);
endmodule

```

Wyniki RTL Viewer:

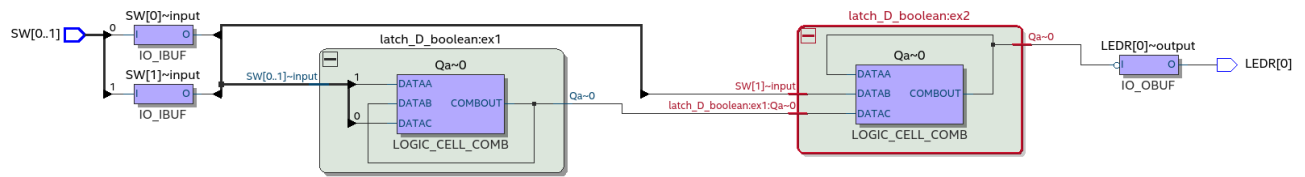


Moduł master_slave_d z użyciem modułu latch_D_boolean z poprzedniego zadania oraz bez użycia dyrektywy keep.

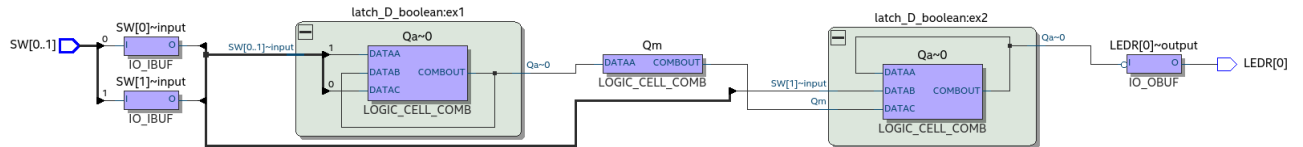


Moduł master_slave_d z użyciem modułu latch_D_boolean z poprzedniego zadania z użyciem dyrektywy keep.

Wyniki Technology Map Viewer:



Moduł master_slave_d z użyciem modułu latch_D_boolean z poprzedniego zadania oraz bez użycia dyrektywy keep.

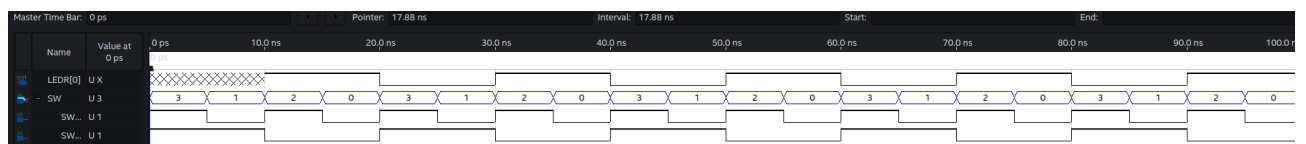


Moduł master_slave_d z użyciem modułu latch_D_boolean z poprzedniego zadania z użyciem dyrektywy keep.

Wyniki symulacji:



Wynik symulacji dla projektu bez użycia dyrektywy keep



Wynik symulacji dla projektu z użyciem dyrektywy keep

Wnioski:

Stworzyliśmy projekt master_slave_D poprzez połączenie dwóch przerzutników typu D. Do opisanego pojedynczego przerzutnika posłużył nam kod modułu latch_D_boolean z zadania 2. Podobnie jak w poprzednich zadaniach, obecność dyrektywy keep skutkuje dodatkowym narzutem bramek, który dostrzec można porównując schematy RTL. Symulator sygnalizuje informacje o błędzie przy wartości początkowej Clk = 1.

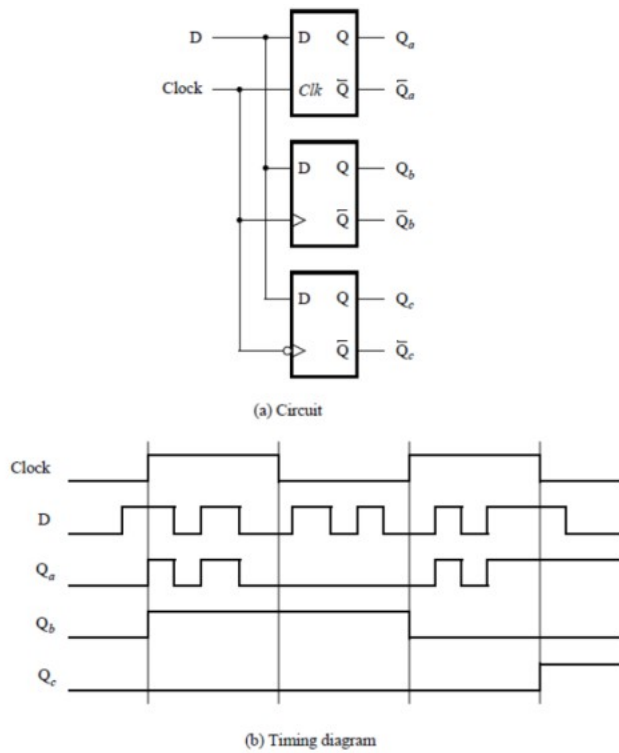
4. Zadanie 4

Utwórz projekt latch_FDDP_FDDN, który implementuje układ z rysunku. 6, a. Sprawdź funkcjonowanie dla diagramu czasowego z rys. 6, b.

Aby opisać zatrzask typu D (Rysunek 6), należy użyć przypisania proceduralnego języka Verilog (rysunek 7). Podobnie należy uczynić, aby opisać projekty przerzutników: FDD_posedge i FDD_negedge.

Wdrożyć projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Qa	LEDR[0]
Qb	LEDR[1]
Qc	LEDR[2]



(a) Schemat
(b) diagram czasowy

Rys. 6. Schemat i diagramy czasowe dla projektu `latch_FDDP_FDDN`

aby opisać zatrząsk typu `D` (rysunek 6), należy użyć przypisania proceduralnego języka Verilog (rysunek 7). Podobnie należy uczynić, aby opisać projekty przerzutników: `FDD_posedge` i `FDD_negedge`.

```
module D_latch (D, Clk, Q);
  input D, Clk;
  output reg Q;
  always @ (D, Clk)
    if (Clk)
      Q = D;
endmodule
```

Rys. 7. Proceduralny styl kodu **Verilog**, opisujący zatrząsk typu **D**

Wdrożyć projekt na płycie z następującymi przypisaniami pinów:

Wyrowadzenia	Piny
D	SW1/01

Kod w języku Verilog:

```
module latch_FDDP_FDDN (  
    input [1:0] SW,  
    output [2:0] LEDR);  
  
    D_latch ext1(SW[0], SW[1], LEDR[0]);  
    FDD_posedge ext2(SW[0], SW[1], LEDR[1]);  
    FDD_negedge ext3(SW[0], SW[1], LEDR[2]);  
  
endmodule
```

Główny moduł.

```
module D_latch (D, clk, Q);  
    input D, clk;  
    output reg Q;  
  
    always @ (D, clk)  
        if (clk)  
            Q = D;  
  
endmodule
```

Kod opisujący zatrzask typu D.

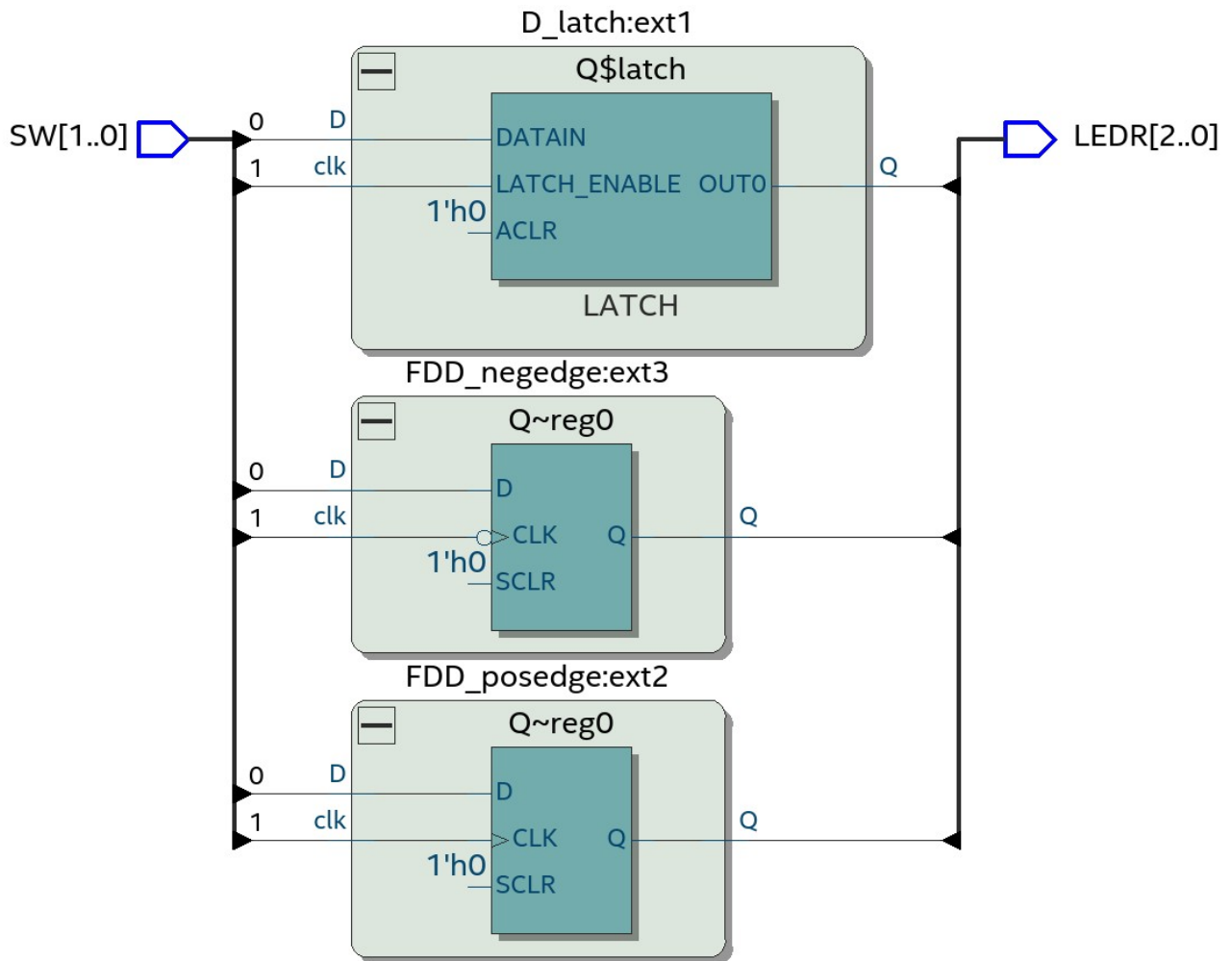
```
module FDD_posedge (D, clk, Q);  
    input D, clk;  
    output reg Q;  
  
    always @ (posedge clk)  
        if (clk)  
            Q = D;  
  
endmodule
```

Kod opisujący przerzutnik sterowany wzrastającym zboczem zegara.

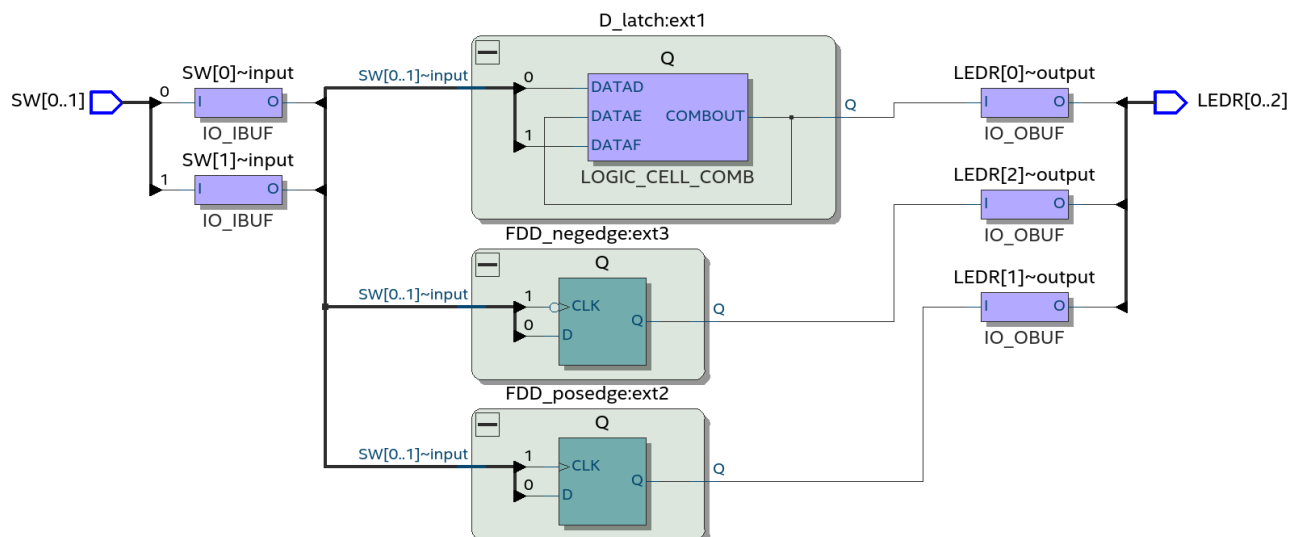
```
module FDD_negedge (D, clk, Q);  
    input D, clk;  
    output reg Q;  
  
    always @ (negedge clk)  
        if (~clk)  
            Q = D;  
  
endmodule
```

Kod opisujący przerzutnik sterowany opadającym zboczem zegara.

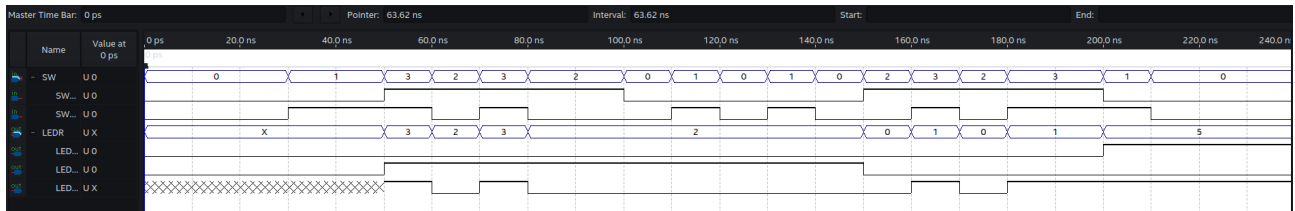
Wyniki RTL Viewer:



Wyniki Technology Map Viewer:



Wyniki symulacji:



Wnioski:

Stosując instrukcje warunkowe `if` oraz dyrektywę `always` opisaliśmy trzy różne zatraski typu D. Pierwszy, `D_latch`, przy stanie wysokim zegara podaje na wyjście (Q_a) sygnał otrzymany na wejściu (D). Drugi, `FDD_posedge`, propaguje sygnał wejściowy na wyjście Q_b tylko przy wzrastającym zboczu zegara, podczas gdy ostatni - `FDD_negedge` - działa analogicznie do `FDD_posedge`, tylko wyzwalany jest podczas opadającego zbocza zegara (jego wyjście to Q_c). Opisanie dwóch ostatnich przerzutników było możliwe, dzięki podaniu do `@Always` jako parametru odpowiednio instrukcji `posedge` i `negedge`.

5. Utwórz projekt `adder_A_B_8_bits`, który dodaje dwie 8-bitowe liczby binarne A i B , wynik jest wyświetlany jako liczba szesnastkowa na ekranach `HEX5` i `HEX4`. Liczby A i B są również wyświetlane w zapisie heksadecymalnym na wyświetlaczach 7-segmentowych: A na `HEX3` i `HEX2`, B na `HEX1` i `HEX0`. Wartości liczb A i B wprowadza się za pomocą przełączników `SW7-0`.

Wskazówka: Użyj multipleksowania podczas wprowadzania liczb A i B , najpierw wprowadź wartość liczby A i zapamiętaj ją w rejestrze, a następnie wprowadź wartość liczby B . Aby zapamiętać numer A , użyj rejestru `reg_N_bits_with_aset` dla N bitów z asynchronicznym resetem. Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
A, B	SW[7:0]
A	HEX3, HEX2
B	HEX1, HEX0
S = A + B	HEX5, HEX4
cout	LEDR[0]
areset	KEY0
clk	KEY1

Kod w języku Verilog:

```

module adder_A_B_8_bits(
    input clk,
    input [7:0] newValue,
    input areset,
    output [0:6] S0, S1, A0, A1, B0, B1,
    output reg cout);

    reg [8:0] S;
    wire [7:0] storedValue;

    reg_N_bits reg1(clk, areset, newValue, storedValue);

    showHexDigit dec_a0(newValue[3:0], A0);
    showHexDigit dec_a1(newValue[7:4], A1);

    showHexDigit dec_b0(storedValue[3:0], B0);
    showHexDigit dec_b1(storedValue[7:4], B1);

    showHexDigit dec_S0(S[3:0], S0);
    showHexDigit dec_S1(S[7:4], S1);

    always @(posedge clk)
    begin
        S = newValue + storedValue;
        if(S > 'hFF)
            cout = 1'b1;
        else
            cout = 1'b0;
    end
endmodule

```

Główny moduł.

```

module showHexDigit (
    input [3:0] SW,
    output reg [0:6] HEX0);

    always @(*)
        case(SW)
            4'b0000: HEX0 = 7'b00000001;
            4'b0001: HEX0 = 7'b10011111;
            4'b0010: HEX0 = 7'b00100101;
            4'b0011: HEX0 = 7'b00000110;
            4'b0100: HEX0 = 7'b10011100;
            4'b0101: HEX0 = 7'b01001100;
            4'b0110: HEX0 = 7'b01000000;
            4'b0111: HEX0 = 7'b00011111;
            4'b1000: HEX0 = 7'b00000000;
            4'b1001: HEX0 = 7'b00000100;
            4'b1010: HEX0 = 7'b00010000;
            4'b1011: HEX0 = 7'b11000000;
            4'b1100: HEX0 = 7'b01100001;
            4'b1101: HEX0 = 7'b10000010;
            4'b1110: HEX0 = 7'b01100000;
            4'b1111: HEX0 = 7'b01110000;
        endcase
endmodule

```

Dekoder szesnastkowy.

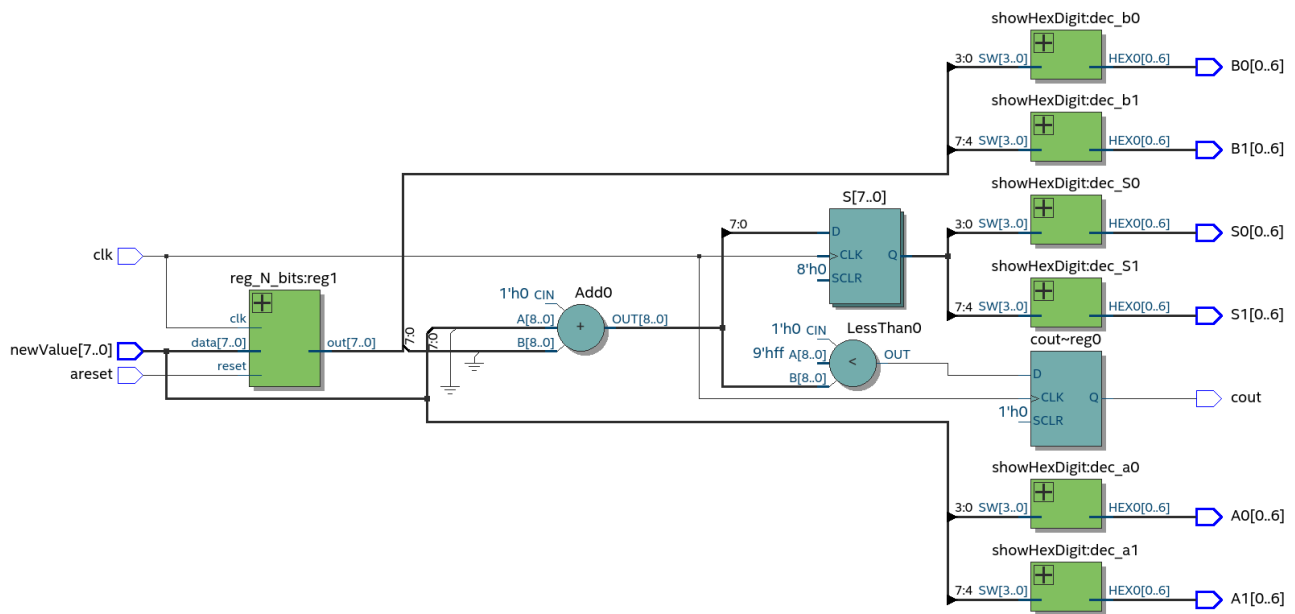
```

module reg_N_bits (
    input clk, reset,
    input wire [7:0] data,
    output reg [7:0] out);
    always @(posedge clk, posedge reset)
        if (reset) out <= 0;
        else out <= data;
endmodule

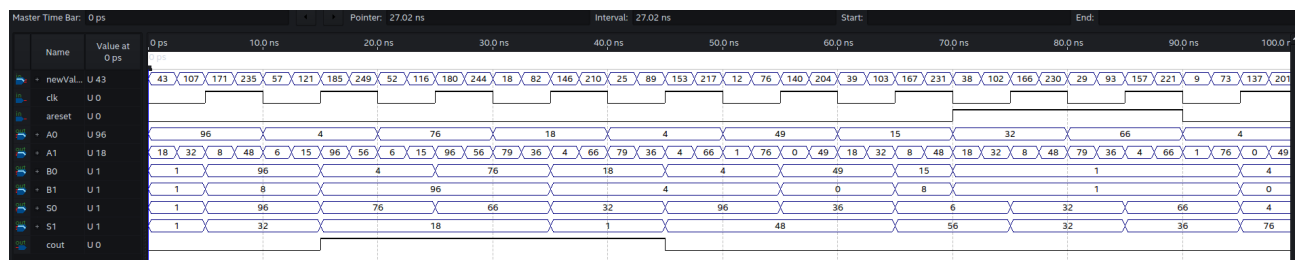
```

Rejestr przechowujący liczbę.

Wyniki RTL Viewer:



Wyniki symulacji:



Wnioski:

Utworzony projekt przy narastającym zboczu zegara zapisuje wartość podaną na wejściu (newValue) do rejestru i wyświetla na wyświetlaczu A. Przy każdym kolejnym narastającym zboczu wartość z rejestru (storedValue) dodawana jest do wartości podanej na wejściu (newValue). Suma wyświetlana jest na wyświetlaczu S. Projekt posiada możliwość wyczyszczenia pamięci rejestru (areset). W przypadku przekroczenia maksymalnej wartości możliwej do wyświetlenia (FF) sygnalizowane jest przeniesienie na diodzie (cout).